

OSSP (Operating Systems and Systems Programming) – 25CS2104E

OSSP LAB – PRACTICAL-2

Aim

To develop a C program using the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another, and to use the `strace` utility to trace and analyze the system calls generated by `cat sample.txt`, thereby understanding the interaction between user space and kernel space.

Algorithm

1. Start the program.
2. Open the source file using `open()` in read-only mode.
3. Open or create the destination file using `open()` in write mode.
4. Read data from the source file using `read()` into a buffer.
5. Write the data from the buffer to the destination file using `write()`.
6. Repeat the read and write operations until the entire file is copied.
7. Close both files using `close()`.
8. Execute `strace cat sample.txt` in the terminal.
9. Observe and analyze the system calls such as `execve()`, `openat()`, `read()`, `write()`, and `close()`.
10. Relate these system calls to the corresponding kernel services and user-space/kernel-space transitions.
11. Stop.

Steps

Step 1

```
bhavana@Bhavana:~$ nano practical2.c
```

Step 2

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int source_fd, dest_fd;
    char buffer[1024];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    source_fd = open(argv[1], O_RDONLY);

    if (source_fd < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest_fd < 0)
    {
        perror("Error opening destination file");
        close(source_fd);
        return 1;
    }

    while ((bytes_read = read(source_fd, buffer, sizeof(buffer))) > 0)
    {
        bytes_written = write(dest_fd, buffer, bytes_read);
    }
}
```

```

while ((bytes_read = read(source_fd, buffer, sizeof(buf-
{
    bytes_written = write(dest_fd, buffer, bytes_read);

    if (bytes_written != bytes_read)
    {
        perror("Error writing to destination file");
        close(source_fd);
        close(dest_fd);
        return 1;
    }
}

if (bytes_read < 0)
{
    perror("Error reading source file");
}

close(source_fd);
close(dest_fd);

printf("File copied successfully.\n");

return 0;
}

```

Step 3

```

bhavana@bhavana:~$ nano practical2.c
bhavana@Bhavana:~$ ls
command_exec  command_exec.c  permission_test.txt  practical2.c  process.c.save  q1.c  readfile  readfile.c  skill1.1  student.txt  task1  task1.c
bhavana@Bhavana:~$

```

Swtep 4

```

bhavana@Bhavana:~$ echo "Hello Linux System Calls" > sample.txt
bhavana@Bhavana:~$ cat sample.txt
Hello Linux System Calls

```

Step 5

```

bhavana@Bhavana:~$ strace -V
strace -- version 6.19
Copyright (c) 1991-2026 The strace developers <https://strace.io>.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Optional features enabled: stack-trace=libunwind stack-demangle m32-mpers mx32-mpers

```

```

bhavana@Bhavana:~$ sudo apt install strace
strace is already the newest version (6.19+ds-0ubuntu5).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 40

```

```

bhavana@bhavana:~$ strace cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffccd29e3f8 /* 25 vars */) = 0
brk(NULL)                               = 0x5c67d06dd000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x720b9b035000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20087, ...}) = 0
mmap(NULL, 20087, PROT_READ, MAP_PRIVATE, 3, 0) = 0x720b9b030000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0....", 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x720b9affd000
mmap(0x720b9b004000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7000) = 0x7
mmap(0x720b9b025000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x720b9b02500
mmap(0x720b9b02c000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2f000) = 0x7
mmap(0x720b9b02e000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x720b9b
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0....", 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=187120, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x720b9afc000
mmap(0x720b9afd3000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x7
mmap(0x720b9afd7000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x720b9aff700
mmap(0x720b9affb000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2c000) = 0x7
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0....", 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=1198376, ...}) = 0
mmap(NULL, 1200152, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x720b9aea9000
mmap(0x720b9aebb000, 655360, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x12000) = 0x
mmap(0x720b9af5b000, 466944, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xb2000) = 0x720b9af5b0
mmap(0x720b9afcd000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x123000) = 0x
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

```

Conclusion

The file-copy operation was successfully implemented using Linux system calls, and the strace utility was used to trace the system calls generated by cat sample.txt. The experiment demonstrated how user-space programs interact with the Linux kernel through system calls for file access, reading, writing, and closing operations.